IEEE 802.1Q-2022, Virtual Local Area Network tagging, store-and-forward versus cut-through, Forwarding Database learning and aging, and switch architecture

1. The big picture

Institute of Electrical and Electronics Engineers (IEEE) 802.1Q-2022 is the base bridge-and-virtual-network standard for Ethernet bridged networks. In the standard's own summary, it specifies how the **Media Access Control (MAC) Service** is supported by bridged networks, the principles of operation of those networks, and the operation of **MAC Bridges** and **Virtual Local Area Network (VLAN) Bridges**, including management, protocols, and algorithms. Current IEEE YANG material also treats **802.1Q-2022** as the base revision that later amendments build on.

802.1Q is not "the VLAN tag document" only. It is the broader rulebook for how Ethernet bridges and switches segment traffic into virtual networks, learn device locations, forward or filter frames, manage priorities, and maintain orderly behaviour in a multi-switch network. VLAN tagging is one visible part of it, but the standard is really about how a bridged Ethernet network behaves as a system.

---

2. Why Ethernet needed 802.1Q in the first place

A plain Ethernet **Local Area Network (LAN)** has no built-in concept of multiple logical broadcast domains sharing the same switching infrastructure. If every port on a switch belonged to one flat Layer 2 domain, then broadcasts, unknown unicasts, and many control effects would spread everywhere. VLANs solve that by letting one physical switch fabric behave like several separate logical LANs. Linux bridge documentation summarizes the practical point well: a VLAN is a logical segmentation of a physical network into multiple isolated broadcast domains.

A VLAN is not "extra metadata for convenience."
A VLAN is a rule for who is considered to be in the same Layer 2 neighborhood.

The tag is just how a switch carries that neighborhood identity across links that transport traffic for multiple VLANs at once.

The real purpose is controlled separation of traffic and forwarding scope.

---

3. What 802.1Q-2022 is really about

A good way to understand 802.1Q-2022 is to divide it into four big ideas:

1. **classification** — deciding what VLAN and priority a received frame belongs to
2. **forwarding** — deciding where a frame should go
3. **filtering** — deciding where a frame must not go
4. **control and management** — deciding how bridges maintain topology, tables, and policy.

That is why the standard reaches far beyond the four-byte tag. IEEE tutorial material on 802.1Q bridge architecture explicitly shows a separation between the **data plane** and the **control plane**, with the data plane built around a **MAC Relay** and bridge ports, and the control side implemented through higher-layer entities or an external agent.

So while trying to understand this topic, let us not start with the tag bits. Start with the question:

How does a switch decide what a frame means, where it belongs, and where it should go next?

---

4. The basic bridge model

At the most basic level, an Ethernet switch is a bridge with multiple ports.

Frames arrive on an ingress port, some forwarding logic inspects them, and the switch decides whether to forward them to one egress port, several egress ports, or none.

IEEE 802.1 tutorial material describes the data plane as a relay with ports, and says bridge forwarding is based on **MAC addresses** and **VLANs**.

The key word here is **relay**.

A switch is not interpreting Internet Protocol routing information at this stage.

It is looking mainly at Layer 2 identifiers: destination MAC address, source MAC address, and VLAN context.

The forwarding decision is therefore not just “which destination MAC address did I see?” but more accurately “which destination MAC address did I see in which VLAN context?”

IEEE tutorial material makes exactly that point by discussing forwarding based on destination MAC plus **VID** context.

That is why VLAN support changes the bridge fundamentally.

Without VLANs, one MAC table may be enough.

With VLANs, the same MAC address can appear in different VLAN contexts, and the forwarding decision must respect that separation.

IEEE YANG material reflects this with support for **Independent VLAN Learning (IVL)** and **Shared VLAN Learning (SVL)** capabilities.

---

5. VLAN tagging from first principles

Now we can talk about the tag itself.

A tagged Ethernet frame carries an extra **four-byte** field inserted between the source MAC address and the original EtherType or length field.

802.1Q is the standard for VLAN tagging in Ethernet networks and that 802.1ad, often called **Q-in-Q**, extends this by stacking multiple VLAN tags in one frame.

Why is the tag inserted there? Because the receiver needs to see the destination and source MAC addresses first, and then very early in parsing it needs to know whether a VLAN tag is present and what VLAN context applies. In practice, that outer tag tells the bridge how to classify and handle the frame before deeper forwarding logic is completed. This is one reason tagged frames are so natural on trunk-style links carrying multiple VLANs.

The outer two bytes of the tag are the **Tag Protocol Identifier (TPID)**, which tells the receiver that this is a VLAN-tagged frame of a certain tag type.

IEEE YANG material identifies the common **Customer VLAN (C-VLAN)** tag as using EtherType **81-00**, and the **Service VLAN (S-VLAN)** tag as using EtherType **88-A8**. The Internet Assigned Numbers Authority registry likewise tracks the corresponding IEEE 802 EtherTypes.

---

6. The fields inside the tag: VID, PCP, and DEI

Inside the tag, the most important subfields are:

- **VLAN Identifier (VID)**
- **Priority Code Point (PCP)**
- **Drop Eligible Indicator (DEI)**.

VLAN Identifier

The **VID** says which VLAN the frame belongs to.

IEEE YANG material repeatedly models VLAN identifiers in the normal user range **1 to 4094**, and also uses 4095 specially for “any” matching in management/classifier contexts, which reflects the usual reserved treatment of the all-ones value.

The important conceptual point is:

The VID is not a "path." It is a membership label.
It tells the switch which logical Layer 2 domain this frame is part of.

The switch then forwards and floods only according to the rules of that VLAN, not the whole box.

Priority Code Point

The **PCP** is a three-bit priority field. IEEE YANG definitions describe the priority range as **0 through 7**, and they also define **Priority Code Point encoding and decoding tables** that map between the PCP and internal priority plus drop-eligibility handling.

PCP is not "just a number in the tag." In reality, it is one of the key bridges between incoming frame marking and internal switch queuing behavior.

A switch can use PCP to map traffic into different internal traffic classes and scheduling treatments.

IEEE YANG material explicitly includes traffic-class mapping and transmission-selection algorithm identities such as **strict priority**, **credit-based shaper**, and **enhanced transmission selection**.

Drop Eligible Indicator

The **DEI** is a one-bit indicator that marks whether a frame is eligible to be dropped under congestion. IEEE YANG text ties PCP handling to both priority and drop-eligible behavior, and explicitly models DEI as the drop-eligible value associated with the tag.

That means a tagged frame does not just say "I belong to VLAN 100." It can also say "inside VLAN 100, treat me as higher or lower urgency, and if congestion becomes severe, I may be drop-eligible." This is a subtle but important distinction between **membership** and **service treatment**.

---

7. What "priority-tagged" means

A useful detail that often confuses learners is the idea of a **priority-tagged** frame. IEEE bridge-management material distinguishes among:

- admit only VLAN-tagged frames
- admit only untagged and priority-tagged frames
- admit all frames.

Why does that matter? Because a frame may carry priority information without being treated as belonging to an ordinary configured VLAN in the same way as a normal tagged VLAN frame. In practice, this lets networks preserve Layer 2 priority semantics even when the

VLAN membership handling is special or derived from port rules. The main teaching point is simply that “tagged” is not a single all-or-nothing bucket; the switch may treat ordinary VLAN-tagged and priority-tagged cases differently at ingress.

---

8. Customer tags, service tags, and Q-in-Q

Now we get to **Q-in-Q**, which is one of the most misunderstood ideas in VLANs.

Q-in-Q means adding another 802.1Q-style tag around an already tagged frame. Linux bridge documentation describes **802.1ad**, also known as QinQ or double VLAN, as stacking multiple VLAN tags inside a single Ethernet frame. Juniper documentation explains the operational purpose well: a provider can prepend a **Service VLAN (S-VLAN)** tag to a customer’s already tagged traffic, which helps carry multiple customers across a provider network even when customers reuse the same VLAN IDs.

This is the real reason Q-in-Q exists:

One tag identifies the customer’s own internal VLAN context.
Another tag identifies how the provider transports that customer traffic.

IEEE YANG material reflects this distinction by defining both **C-VLAN** and **S-VLAN** tag types, with **81-00** for the customer tag and **88-A8** for the service tag.

So the “Q-in-Q” name is a shorthand. The actual concept is **tag stacking** so that one administrative domain can encapsulate another domain’s VLAN-marked traffic without destroying the inner tagging semantics.

---

9. Why Q-in-Q is useful

Suppose two customers both use VLAN 100 inside their own sites. A service provider cannot just bridge both customers’ VLAN 100 traffic into one flat provider domain, because the provider would lose customer separation. Q-in-Q solves that by wrapping the customer tag with a provider service tag. Then the provider switch fabric can forward based on the outer tag, while the customer’s inner tag remains intact for delivery back to the customer side.

That means the outer tag and inner tag do not have to mean the same thing. The outer tag is often “transport context.” The inner tag is often “customer-local VLAN meaning.” That is why stacked VLAN tagging is powerful: it separates **who owns the traffic’s inner structure** from **who owns the transport path**.

---

10. Port-based reality: access behaviour and trunk behaviour

Even though the standard is broader than common enterprise vocabulary, it is still useful pedagogically to explain two familiar behaviours.

An ingress port can receive an untagged frame and assign it to a VLAN based on port configuration. IEEE tutorial material notes that when the outermost EtherType is not the bridge-component type, the frame is assigned a VLAN by the reception port. IEEE YANG material also includes **Port VLAN Identifier (PVID)**-related capabilities and ingress filtering behavior.

A different port can carry frames for multiple VLANs by leaving tags in place across the link. That is the reason tagged multi-VLAN inter-switch links exist at all. Linux bridge documentation summarizes this behavior when it says VLAN filtering on a bridge causes forwarding decisions to depend on both destination MAC address and VLAN tag.

So a very useful mental model is:

- on some ports, VLAN identity is supplied by the **port's classification rule**
- on other ports, VLAN identity is supplied by the **frame's explicit tag**.

---

11. Ingress filtering and acceptable frame types

A switch does not have to accept every possible incoming tag pattern on every port. IEEE bridge YANG material explicitly models **acceptable frame types** and **ingress filtering** per port. A port can be configured to admit only VLAN-tagged frames, only untagged and priority-tagged frames, or all frames; ingress filtering can also be enabled so that frames for VLANs not permitted on that port are discarded.

This matters because VLANs are not just an egress tagging trick. The switch enforces VLAN policy on the way in too. If a frame arrives with a VLAN context that the port is not allowed to participate in, the correct action is often to drop it right there rather than let it enter the forwarding pipeline.

That is one reason VLAN misconfiguration can look so strange in practice: the frame may not even make it into normal forwarding logic if ingress classification or ingress filtering rejects it first.

---

12. The Forwarding Database: what it is really for

The **Forwarding Database (FDB)** is the table the bridge uses to remember where known MAC destinations are located. IEEE 802.1Q bridge-management material describes the filtering database as containing the information used by the forwarding process in deciding through which ports frames should be forwarded. It also exposes FDB size, counts of static entries, counts of dynamic entries, and the aging time for dynamically learned entries.

The most important conceptual point is:

The FDB is the switch's memory of "where I last saw this source address in this VLAN context."

Without that memory, the switch would have to flood almost everything. With that memory, it can unicast known destinations to the correct egress port and avoid unnecessary traffic elsewhere.

---

13. How MAC learning works from first principles

When a frame enters a switch, the switch sees both a **source MAC address** and a **destination MAC address**. The source address is extremely valuable because it tells the switch, "if someone later needs to send traffic back to this source, it can probably be reached through this ingress port and this VLAN context." IEEE tutorial material explicitly includes **source address learning** as part of the station-location topology view, and Linux bridge documentation says a port learns source MAC addresses from received traffic and refreshes dynamic FDB entries based on incoming traffic.

That leads to the classic learning sequence:

1. a frame arrives on port X in VLAN Y
2. the switch records that source MAC S is reachable on port X in VLAN Y
3. the switch looks up the destination MAC D in the relevant FDB context
4. if D is known, it forwards to the learned egress port
5. if D is unknown, it floods within the VLAN's permitted scope.

Two key parts of this story: learning from source MAC addresses and flooding unknown unicasts when there is no FDB entry.

---

14. Why VLAN context must be part of learning

A switch cannot always treat a MAC address as globally unique across all VLANs for forwarding purposes. IEEE YANG material exposes **Independent VLAN Learning (IVL)** and **Shared VLAN Learning (SVL)** capability flags, and IVL is the default-capable model in the bridge YANG view.

For understanding purposes, the important idea is:

- with **IVL**, the same MAC address can be learned separately in different VLAN contexts
- with **SVL**, learning can be shared more broadly across VLAN contexts, depending on the bridge model.

A VLAN-aware bridge usually does not learn in one flat universe. It learns in a VLAN-aware context because forwarding must preserve VLAN separation.

---

15. Dynamic entries, static entries, and aging

IEEE bridge-management material distinguishes **static** and **dynamic** filtering entries and gives a default **aging time** of **300 seconds** for dynamically learned forwarding information. It also describes the aging time as the timeout period for aging out dynamically learned information.

This is fundamental to how switches remain correct over time. A host can move. A cable can be moved. A device can disappear. If learned entries never expired, the switch could continue sending traffic toward the wrong port indefinitely. Aging solves that by making learned state provisional rather than eternal.

So the logic is:

- **dynamic entries** are learned automatically from traffic and refreshed by traffic
- **static entries** are configured deliberately and are not dependent on ordinary traffic refresh
- **aging** removes stale dynamic knowledge so the FDB reflects current topology rather than ancient history.

---

16. What happens when the destination is unknown

If the destination MAC address does not have an FDB entry, the switch cannot confidently send the frame to one egress port. In that case it performs **unknown unicast flooding** within the allowed VLAN scope. Linux bridge documentation says unicast traffic for which there is no FDB entry will be flooded toward a port when unknown-unicast flooding is enabled, and it also notes that forwarding with VLAN filtering depends on both destination MAC and VLAN tag.

This is a critical point because it explains why a “quiet” network can suddenly generate wider traffic spread when a learned entry ages out or a destination is first contacted. Unknown traffic must initially be treated more like a discovery problem. Once a return frame arrives, source learning can populate the FDB and future forwarding becomes more selective.

So flooding is not necessarily a fault. It is often the bridge’s safe fallback when it does not yet know a precise destination location.

---

17. Store-and-forward versus cut-through: the underlying question

Now move from logical forwarding to the forwarding **timing** model inside the switch.

The real question is:

Does the switch wait for the entire frame to arrive before forwarding, or does it start forwarding as soon as it knows enough?

That is the difference between **store-and-forward** and **cut-through** switching. In cut-through mode the switch forwards the data before it has completed receiving the entire frame, while in store-and-forward mode it stores the frame until the entire frame has been received and checked for **Cyclic Redundancy Check (CRC)** errors.

That description sounds simple, but the implications are deep. The choice changes forwarding latency, error handling, buffering behavior, and how the switch copes with speed mismatches and congestion.

---

18. Store-and-forward from first principles

In **store-and-forward**, the switch receives the whole frame into a buffer, checks the frame integrity, and only then decides to place it onto the egress port. Cisco documentation says store-and-forward checks each frame for CRC errors before forwarding and waits until the entire frame has been received and checked.

Why would you want that? Because it is the safer, cleaner forwarding model.

The switch can:

- verify the frame is not corrupted
- handle ingress and egress speed mismatches more gracefully
- make forwarding and congestion decisions with full knowledge of the received frame
- avoid propagating frames that already failed integrity checks.

So the central idea is:

store-and-forward buys correctness and robustness by paying extra latency.

---

19. Cut-through from first principles

In **cut-through**, the switch starts forwarding before the full frame has arrived. Cisco documentation says the switch can forward as soon as it has read the destination details in the packet header, which makes it faster than store-and-forward.

Why would that help? Because the switch does not wait for the tail of the frame before starting egress transmission. That reduces port-to-port latency, especially for larger frames.

But the price is obvious:

if the frame later turns out to be bad, some or all of it may already have been forwarded. Store-and-forward drops packets with Frame Check Sequence errors, while cut-through is about faster forwarding based on early header information rather than full-frame verification first.

So the simple comparison is:

- **cut-through** favors latency reduction
- **store-and-forward** favors validation and robustness.

---

20. Why cut-through is not “instant”

It is incorrect to imagine cut-through as forwarding right after the first six destination bytes. Real switches still need enough early header information to make a correct forwarding decision. On tagged Ethernet, that can include not just the destination MAC address but also the outer tag context, because forwarding is VLAN-aware. Since IEEE 802.1Q forwarding depends on destination MAC and VLAN context, a cut-through bridge still has to parse enough of the frame to know the relevant forwarding domain before it can safely choose egress treatment. This is an inference from the combination of cut-through behavior and VLAN-aware forwarding rules.

So the right statement is not “cut-through forwards after six bytes.” It is:

cut-through forwards after the switch has enough header information to make a forwarding decision, without waiting for the whole frame and the final integrity check.

---

21. When store-and-forward is operationally preferable

Store-and-forward becomes especially attractive when:

- ingress traffic may exceed egress capacity
- the switch must drop bad frames early
- you care more about predictable validation than absolute minimum latency.

Store-and-forward is activated automatically for oversubscribed conditions where ingress rate exceeds egress-port switching capacity. That is a practical reminder that some traffic conditions naturally favor buffering and validation over ultra-low-latency forwarding. (Cisco)

So in a real network, the question is not “which mode is universally better?” It is “what matters more here: lowest possible forwarding latency, or stronger buffering/integrity behavior?”

---

22. Switch architecture overview: the internal blocks

Now we can connect the logical bridge model to an implementation view.

IEEE tutorial material separates the **control plane** from the **data plane** and says the data plane is made up of a **MAC Relay** and at least two ports. That is the architectural heart of a switch.

A simplified implementation view looks like this:

1. **ingress port logic** receives the frame
2. **parser / classifier** determines tag presence, VLAN context, priority, and other metadata
3. **FDB lookup** determines whether destination is known
4. **policy / filtering logic** decides whether forwarding is allowed in that VLAN and on those ports
5. **buffering and queuing** prepare the frame for one or more egress ports
6. **egress port logic** transmits the frame, possibly with tag insertion, removal, or rewrite according to port policy. This structure is an implementation-oriented explanation built on the standard's relay-and-port architecture and its VLAN-aware bridge model.

The main point is that a switch is not "just a table lookup." It is a pipeline:

receive → classify → learn → look up → filter → queue → transmit.

---

23. Control plane versus data plane

IEEE tutorial material is especially helpful here because it says the bridge architecture separates the control plane from the data plane and allows higher-layer entities or even an external agent to provide control.

That means the forwarding pipeline does not decide everything by itself. The control side determines things like:

- which VLANs exist
- which ports belong to which VLANs
- which topology-control method is active
- what static forwarding or filtering policy is installed
- whether learning, flooding, or restrictions are enabled.

This is a very important conceptual point. The data plane forwards frames in real time, but the control plane determines the rules under which those forwarding decisions are made.

---

24. Why source learning and flooding are both necessary

A bridge cannot learn where a destination lives until it sees that destination as a **source** in some frame. IEEE tutorial material explicitly includes source-address learning as part of station-location knowledge, and Linux bridge documentation explains that learning comes from source MAC addresses while unknown unicasts are flooded.

That means learning and flooding are paired behaviors:

- learning reduces future flooding
- flooding is the fallback when learning is not yet available.

This is one of the most foundational things students should understand about Ethernet switching. Many troubleshooting symptoms make sense only when you remember that the switch learns from sources, not from destinations.

---

25. Why aging is not optional

If a device moves from one port to another, or if a topology change makes the old learned path invalid, stale FDB entries become dangerous. IEEE bridge-management material sets aging time for dynamically learned forwarding information and exposes that aging parameter directly.

Without aging, the switch would remain overconfident about old information. Traffic could keep being sent to the wrong port long after reality changed. With aging, old dynamic entries time out unless refreshed by continued source traffic. Linux bridge documentation explicitly says incoming traffic refreshes dynamic FDB entries.

So aging is not a housekeeping footnote. It is part of how a bridge stays correct in a changing network.

---

26. The simplest mental model of a VLAN-aware switch

A VLAN-aware switch is a box that learns **which source MAC addresses appear on which ports inside which VLAN context**, and then uses that knowledge to forward future destination traffic only where it belongs, while tagging, untagging, or stacking tags according to port and service rules.

That model naturally includes:

- access-like behavior, where a port assigns VLAN context
- trunk-like behavior, where the frame carries VLAN context explicitly

- provider-tagging behavior, where Q-in-Q stacks transport context around customer context
- dynamic learning and aging in the Forwarding Database.

---

27. Common misconceptions

Misconception 1: IEEE 802.1Q is only the four-byte VLAN tag format.
No. The standard is much broader: it specifies how the MAC service is supported by bridged networks and the operation of MAC bridges and VLAN bridges, including management, protocols, and algorithms.

Misconception 2: The VLAN Identifier alone tells the switch everything it needs to know.
No. The tag also carries **Priority Code Point** and **Drop Eligible Indicator** information, and the bridge can map those into internal priority and traffic-class behavior.

Misconception 3: Q-in-Q is just "two tags for no particular reason."
No. It is used so one administrative domain can transport another domain's tagged traffic, often to keep customer VLAN meaning intact while the provider uses its own transport VLAN context.

Misconception 4: The Forwarding Database is only keyed by MAC address.
Not in a VLAN-aware world. IEEE 802.1Q management models support VLAN-aware learning approaches such as Independent VLAN Learning, and Linux bridge forwarding with VLAN filtering depends on destination MAC address and VLAN tag together.

Misconception 5: Cut-through is always better because it is faster.
No. It is faster because it forwards before the full frame and its Cyclic Redundancy Check are complete, but that means it sacrifices some integrity-checking behavior that store-and-forward provides.

Misconception 6: Aging is just optional cleanup.
No. Aging is what prevents dynamic learning from becoming stale and wrong after movement or topology change. IEEE 802.1Q management models explicitly expose aging time for dynamically learned forwarding information.

---

28. Final take-away

IEEE 802.1Q-2022 is the foundational bridge-and-VLAN standard for Ethernet switched networks. **VLAN tagging** adds not only a **VLAN Identifier** but also service-treatment hints like **Priority Code Point** and **Drop Eligible Indicator**. **Q-in-Q** extends the idea by stacking customer and service tags so different administrative domains can coexist cleanly. A switch's **Forwarding Database** is the memory that ties source addresses to ports and VLAN context, and **learning plus aging** keep that memory both useful and current. Finally, **store-and-forward** and **cut-through** are two different forwarding philosophies inside the switch

pipeline: one waits for the whole frame and checks it, while the other forwards earlier once enough header information is known.